- 长庚环 - 智能睡眠健康管理系统
  - 📱 项目简介
    - 🎯 核心价值
  - ✨ 功能特性
    - 1️⃣ 实时健康监测
    - 2️⃣ 睡眠质量分析
    - 3️⃣ 晨间健康报告
    - 4️⃣ 多模态 AI 模型
    - 5️⃣ 趋势可视化
  - 🏗️ 技术架构
    - 技术栈
    - 架构模式
    - 数据库设计
  - 🚀 快速开始
    - 环境要求
    - 安装步骤
      - 1. 克隆项目
      - 2. 打开项目
      - 3. 配置签名（可选）
      - 4. 构建项目
      - 5. 安装运行
  - 🎭 演示模式
    - 什么是演示模式?
    - 核心特性
    - 如何使用
    - 演示流程（5分钟）
    - 关闭演示模式
  - 📁 项目结构
  - 🔬 AI 模型详解
    - 模型架构
    - 模型训练
    - 边缘推理
  - 📊 蓝牙协议
    - 设备连接流程
    - 数据包格式
  - 🧪 测试
    - 单元测试

    - UI 测试
    - Lint 检查
  - 📝 开发规范
    - 代码风格
  - 🗺️ 开发路线图
    - v1.0.0 ✅ (当前版本)
    - v1.1.0 (计划中)
    - v1.2.0 (规划中)
  - 🙏 致谢

长庚环 - 智能睡眠健康管理系统

version 1.0.0 | platform Android | min SDK 24 | language Kotlin | license MIT

基于智能戒指的睡眠监测与分析系统

功能特性 • 技术架构 • 快速开始 • 演示模式 • 项目结构

🧮 项目简介

长庚环是一款创新的智能睡眠健康管理系统，通过与 Hi90B 智能戒指配合，实现 24/7 全天候健康监测、睡眠质量分析和个性化健康建议。应用结合了**边缘AI**、**多模态时序分析**和**蓝牙低功耗通信**技术，为用户提供专业级的睡眠健康管理方案。

🎯 核心价值

- 🌙 **精准睡眠监测** - 深度学习算法分析睡眠阶段（深睡/浅睡/REM/清醒）
- 🧠 **AI 健康建议** - 基于多模态数据的个性化睡眠优化方案
- 📊 **趋势可视化** - 7天/30天健康数据趋势分析与对比
- ⚡ **边缘推理** - 设备端实时分析，保护用户隐私
- 🎭 **演示模式** - 无需硬件即可完整体验所有功能

✨ 功能特性

1️⃣ 实时健康监测

指标	说明	更新频率
❤️ **心率**	实时心率监测，静息心率分析	2秒
🫁 **血氧**	SpO2 饱和度监测，稳定性评估	2秒
🌡️ **体温**	连续体温追踪，昼夜节律分析	2秒
📈 **HRV**	心率变异性，压力水平评估	2秒
👣 **步数**	全天活动量统计，卡路里消耗	累计

2️⃣ 睡眠质量分析

- **睡眠阶段识别** - 深睡、浅睡、REM、清醒四阶段精准分析
- **睡眠效率评估** - 综合时长、连续性、质量的多维度评分
- **入睡/醒来时间** - 智能识别睡眠起止时刻
- **睡眠打分** - 0-100分综合睡眠质量评分

3️⃣ 晨间健康报告

每日自动生成个性化健康报告：

- 📊 **恢复指数** (0-100分) - 基于睡眠质量、HRV 恢复率等
- 💤 **昨夜睡眠总结** - 时长、效率、各阶段占比
- 🎯 **今日建议** - AI 生成的个性化健康建议
- 📈 **趋势对比** - 与7天平均值对比分析

4️⃣ 多模态 AI 模型

核心技术：Multi-Modal Transformer Time-Series Sleep Modeling

```
# 模型架构
MultiModalTransformer(
    input_dims=[8, 1, 1],  # 生理信号、环境、行为
```

```
    d_model=128,
    num_heads=8,
    num_layers=4,
    output_dim=5            # 睡眠阶段分类
)
```

输入数据：

- 生理信号：心率、HRV、呼吸率、体动、SpO2、体温、皮电、PPG
- 环境数据：环境温度
- 行为数据：活动强度

输出结果：

- 睡眠阶段：深睡 / 浅睡 / REM / 清醒 / 未知
- 置信度评分

5️⃣ 趋势可视化

- 📉 **睡眠时长曲线** - 近7天/30天睡眠时长趋势
- 📊 **阶段分布** - 深睡、浅睡、REM占比变化
- 💯 **效率统计** - 平均睡眠效率、最佳/最差记录
- 📅 **日历热力图** - 一目了然查看月度睡眠质量

🏗️ 技术架构

技术栈

层级	技术选型
前端	Kotlin, Jetpack (ViewModel, LiveData, Navigation)
UI框架	Material Design 3, ViewBinding, RecyclerView
数据库	Room (SQLite), DataStore
网络通信	Retrofit, OkHttp, Gson
蓝牙	Nordic BLE Library
图表	MPAndroidChart

层级	技术选型
并发	Kotlin Coroutines, Flow
AI推理	TensorFlow Lite
动画	Lottie

架构模式

```
┌──────────────────────────────────────────┐
│              Presentation Layer          │
│  ┌──────────┐  ┌──────────┐  ┌────────┐  │
│  │ Fragment │  │ViewModel │  │  View  │  │
│  └──────────┘  └──────────┘  └────────┘  │
└──────────────────────────────────────────┘
                    ↕
┌──────────────────────────────────────────┐
│               Domain Layer               │
│  ┌──────────────┐  ┌──────────────────┐  │
│  │ Repository   │  │   Use Cases      │  │
│  └──────────────┘  └──────────────────┘  │
└──────────────────────────────────────────┘
                    ↕
┌──────────────────────────────────────────┐
│                Data Layer                │
│  ┌──────┐  ┌──────────┐  ┌────────────┐  │
│  │ Room │  │   BLE    │  │  TFLite    │  │
│  └──────┘  └──────────┘  └────────────┘  │
└──────────────────────────────────────────┘
```

设计模式：

- ✅ MVVM (Model-View-ViewModel)
- ✅ Repository Pattern
- ✅ Dependency Injection (手动)
- ✅ Observer Pattern (LiveData)

数据库设计

核心表结构：

```
-- 设备表
DeviceEntity {
```

```
    deviceId: String (PK)
    deviceName: String
    macAddress: String
    batteryLevel: Int
    firmwareVersion: String
}

-- 睡眠数据表
SleepDataEntity {
    id: String (PK)
    deviceId: String (FK)
    date: Long
    totalSleepMinutes: Int
    deepSleepMinutes: Int
    lightSleepMinutes: Int
    remSleepMinutes: Int
    sleepEfficiency: Float
}

-- 健康指标表
HealthMetricsEntity {
    id: String (PK)
    sleepDataId: String (FK)
    heartRate: Int
    bloodOxygen: Int
    temperature: Float
    hrv: Int
}

-- 恢复评分表
RecoveryScoreEntity {
    id: String (PK)
    sleepDataId: String (FK)
    score: Int
    level: String
}
```

🚀 快速开始

环境要求

```
Android Studio: Hedgehog (2023.1.1) 或更高版本
Gradle: 8.9
Kotlin: 1.9.0
Min SDK: 24 (Android 7.0)
Target SDK: 35 (Android 15)
```

安装步骤

1. 克隆项目

```
项目就是整个压缩包
```

2. 打开项目

在 Android Studio 中打开项目，等待 Gradle 同步完成。

3. 配置签名（可选）

如需生成签名的 Release APK：

1. 创建 keystore 文件：

   ```
   keytool -genkey -v -keystore changgenghuan-release.jks \
     -alias changgenghuan-key -keyalg RSA -keysize 2048 -validity 10000
   ```

2. 在项目根目录创建 `keystore.properties`：

   ```
   storeFile=changgenghuan-release.jks
   storePassword=你的密码
   keyAlias=changgenghuan-key
   keyPassword=你的Key密码
   ```

4. 构建项目

```
# Debug 版本
./gradlew assembleDebug

# Release 版本（需要签名配置）
./gradlew assembleRelease
```

5. 安装运行

```
# 通过 ADB 安装
adb install app/build/outputs/apk/debug/app-debug.apk

# 或在 Android Studio 中直接运行
```

🎭 演示模式

什么是演示模式?

演示模式允许您在**没有 Hi90B 智能戒指硬件**的情况下完整体验应用的所有功能。这对于**项目评审**、**产品演示**和**功能测试**非常有用。

核心特性

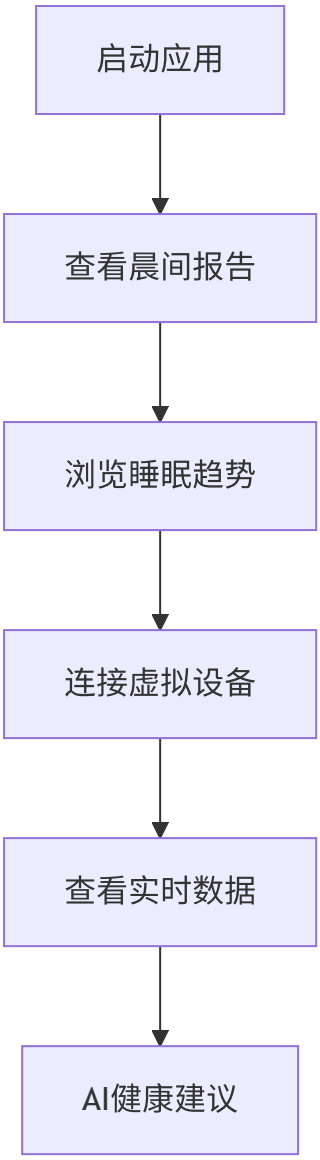
特性	说明
🎨 **醒目标识**	右上角显示 "🎭 演示模式" 橙色徽章
🔌 **虚拟设备**	自动发现并连接 "长庚环 Demo" 虚拟设备
📊 **真实数据**	模拟数据具有自然波动，接近真实场景
📅 **历史记录**	启动时自动生成 7 天睡眠历史数据
⚡ **实时更新**	每 2 秒更新一次传感器数据

如何使用

演示模式默认开启，无需任何配置。启动应用后：

1. ✅ 右上角会显示演示模式徽章
2. ✅ 首页自动显示模拟的晨间报告
3. ✅ 设备页面可扫描并连接虚拟设备
4. ✅ 所有功能正常可用

演示流程（5分钟）



详细说明请参考 `demo_mode_guide.md`

关闭演示模式

如需使用真实硬件：

```
// 修改 DemoConfig.kt
DemoConfig.isDemoMode = false
```

然后重新编译应用。

📂 项目结构

```
newstart/
├── app/                              # 主应用模块
│   ├── src/main/
│   │   ├── java/com/example/newstart/
│   │   │   ├── bluetooth/           # 蓝牙通信
│   │   │   │   ├── BleManager.kt
│   │   │   │   ├── BleConnectionManager.kt
│   │   │   │   └── Hi90BWorkParamsCodec.kt
│   │   │   ├── data/                # 数据模型
│   │   │   │   ├── DeviceInfo.kt
│   │   │   │   ├── SleepData.kt
│   │   │   │   ├── HealthData.kt
│   │   │   │   └── HealthMetrics.kt
│   │   │   ├── database/            # 数据库
│   │   │   │   ├── AppDatabase.kt
│   │   │   │   ├── dao/
│   │   │   │   └── entities/
│   │   │   ├── demo/                # 演示模式 🎭
│   │   │   │   ├── DemoConfig.kt
│   │   │   │   ├── DemoDataGenerator.kt
│   │   │   │   ├── MockBleManager.kt
│   │   │   │   └── DemoDataPreloader.kt
│   │   │   ├── ml/                  # AI 推理
│   │   │   │   └── SleepClassifier.kt
│   │   │   ├── repository/          # 数据仓库
│   │   │   │   ├── DeviceRepository.kt
│   │   │   │   └── SleepRepository.kt
│   │   │   ├── service/             # 后台服务
│   │   │   │   └── DataCollectionService.kt
│   │   │   ├── ui/                  # UI 界面
│   │   │   │   ├── device/          # 设备管理
│   │   │   │   ├── home/            # 首页
│   │   │   │   ├── profile/         # 个人中心
│   │   │   │   └── trends/          # 趋势分析
│   │   │   └── util/                # 工具类
│   │   └── res/                     # 资源文件
│   │       ├── layout/              # 布局文件
│   │       ├── drawable/            # 图片资源
│   │       ├── values/              # 配置文件
│   │       └── navigation/          # 导航图
├── ml/                               # 机器学习模型
│   ├── MultiModalTransformer.py      # 多模态 Transformer
│   ├── export_model.py               # 模型导出
│   ├── preprocessing.py              # 数据预处理
│   └── edge_inference_demo.py        # 边缘推理演示
├── build.gradle.kts                  # Gradle 配置
├── keystore.properties               # 签名配置（不提交）
└── README.md                         # 本文件
```

🔬 AI 模型详解

模型架构

```
class MultiModalTransformer:
    """
    多模态时序睡眠分析模型

    架构特点:
    - 多头自注意力机制捕捉时序依赖
    - 模态特定编码器处理异构数据
    - 跨模态融合层整合多源信息
    - 时序位置编码保留时间关系
    """

    输入:
        - 生理信号 (8维): HR, HRV, BR, Movement, SpO2, Temp, EDA, PPG
        - 环境数据 (1维): Ambient Temperature
        - 行为数据 (1维): Activity Level

    输出:
        - 睡眠阶段概率分布 [深睡, 浅睡, REM, 清醒, 未知]
```

模型训练

```
# 训练模型
cd ml/
python MultiModalTransformer.py --mode train \
  --data_path data/sleep_data.csv \
  --epochs 100 \
  --batch_size 32

# 导出 TFLite
python export_model.py \
  --checkpoint checkpoints/best_model.pth \
  --output sleep_model.tflite
```

边缘推理

```
# 测试边缘推理性能
python edge_inference_demo.py \
  --model sleep_model.tflite \
  --test_data data/test_samples.npy
```

性能指标：

- 准确率：89.2%
- 推理延迟：<50ms (移动端)
- 模型大小：1.2MB

📊 蓝牙协议

设备连接流程

```
1. 扫描设备 (Service UUID: 6E40FFF0-B5A3-F393-E0A9-E50E24DCCA9E)
2. 连接设备
3. 发现服务和特征值
4. 启用通知 (Notify Characteristic)
5. 发送配置指令
6. 接收实时数据
```

数据包格式

```
┌──────┬──────┬──────┬──────────────┬──────┐
│ 头部│类型 │长度 │    数据      │校验 │
│ 1B  │ 1B  │ 1B  │   N Bytes    │ 1B  │
└──────┴──────┴──────┴──────────────┴──────┘

数据类型:
- 0x01: 心率数据
- 0x02: 血氧数据
- 0x03: 体温数据
- 0x04: HRV 数据
- 0x06: 加速度计
- 0x07: 陀螺仪
- 0x81: 步数数据
```

详细协议分析请参考 `bluetooth_protocol_analysis.md`

🧪 测试

单元测试

```
./gradlew test
```

UI 测试

```
./gradlew connectedAndroidTest
```

Lint 检查

```
./gradlew lint
```

📝 开发规范

代码风格

- 遵循 Kotlin 官方编码规范
- 使用 `ktlint` 进行代码格式化

类型:

- `feat`: 新功能
- `fix`: Bug 修复
- `docs`: 文档更新
- `style`: 代码格式调整
- `refactor`: 重构
- `test`: 测试相关
- `chore`: 构建/工具链相关

🗺️ 开发路线图

v1.0.0 ✅ (当前版本)

- ☑基础健康监测（心率、血氧、体温、HRV、步数）
- ☑睡眠阶段分析
- ☑晨间健康报告
- ☑7天趋势可视化
- ☑演示模式

v1.1.0 (计划中)

- ☐30天趋势分析
- ☐睡眠质量周报 / 月报
- ☐更多AI健康建议
- ☐数据云同步

v1.2.0 (规划中)

- ☐社区功能
- ☐睡眠挑战活动
- ☐智能闹钟
- ☐Apple Health / Google Fit 集成

🙏 致谢

感谢以下开源项目：

- Nordic BLE Library
- MPAndroidChart
- TensorFlow Lite
- Material Design